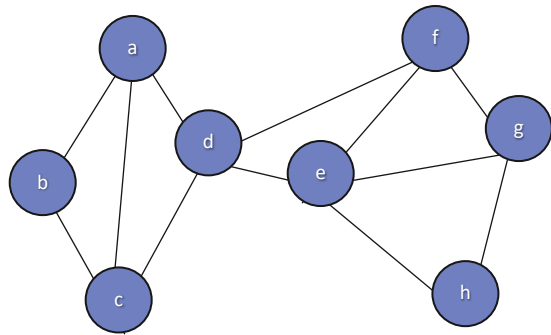


Графовые нейросети и их применение

Обозначения



$G = (V, E)$, где

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

$|V|$ - количество вершин

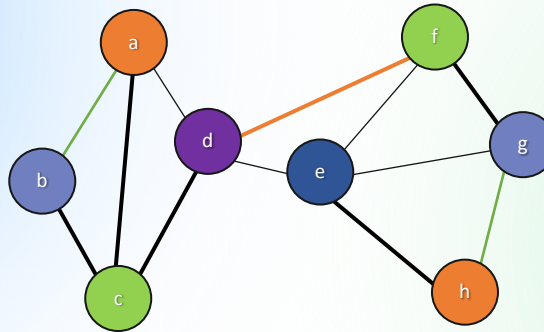
$|E|$ - количество ребер

u, v - вершины

d_u - степень вершины u

e_{uv} - признаки ребра, соединяющие вершины u, v

Гетерогенные графы



Множество узлов разделено на несколько типов

$$\phi(v) \in A$$

Множество рёбер разделено на типы

$$\psi(e) \in R$$

Ограничения классических GNN

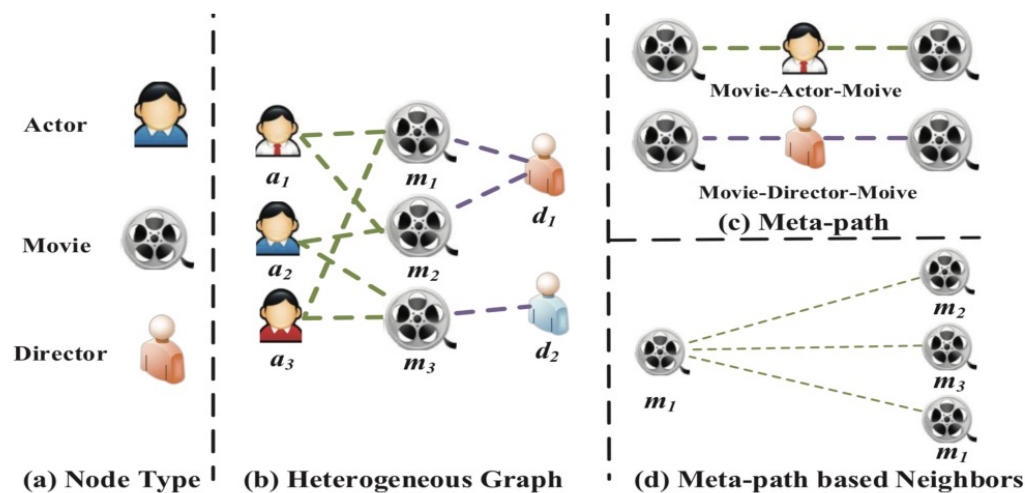
Классические GNN - GCN, GraphSAGE, GAT - имеют ключевое ограничение: они используют один набор весов для всех соседей.

А это означает:

- GNN не различает, кто был соседом
User → Item или *Item → Category*
- GNN не понимает, что разные ребра передают разную информацию
(покупка ≠ просмотр ≠ клик)
- GNN не может моделировать семантические пути, например
User → Item → Category → Item

Примеры

- Рекомендательные системы
- Fraud detection
- Научные и информационные графы (citation / knowledge graphs)
- Социальные графы



Источник: <https://arxiv.org/pdf/1903.07293>

Семантика в гетерогенных графов

Основная идея гетерогенных графов — семантика важнее структуры.

Два ребра могут выглядеть структурно одинаково ($u \rightarrow v$), но нести абсолютно разный смысл.

Семантика задаётся:

- типом узла
- типом ребра
- направлением
- ролью в метапути
- наличием контекстов (время, текст, атрибуты)

Гетерогенные графы - не “несколько типов”

Есть распространённое заблуждение:

«Можно же просто добавить one-hot тип ребра, зачем отдельные модели?»

- **Проблема 1. Разная семантика → разные параметры**

Для отношения “пользователь → товар” и “товар → категория”

нужны разные весовые матрицы.

- **Проблема 2. Разные типы узлов имеют разные пространства**

Embedding для узлов разных типов нельзя сравнивать напрямую.

Сначала их надо привести в общее пространство.

- **Проблема 3. Агрегация должна зависеть от типа соседа**

Например, вклад категории должен весить иначе, чем вклад другого товара.

- **Проблема 4. Семантические пути (метапуть) формируют структуру**

User → Item → Category → Item

формирует паттерн, который обычный GNN никогда не увидит.

Направления методов

В итоге возникает необходимость в моделях, которые:

1. Учитывают типы рёбер (relation-level)

R-GCN — Relation GCN

Каждый тип ребра имеет свою матрицу весов.

2. Учитывают семантические пути (meta-path attention)

HAN — Heterogeneous Attention Network

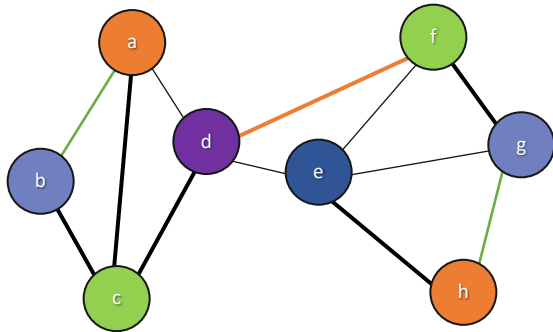
Агрегация = структурное внимание + семантическое внимание.

3. Учитывают оба аспекта одновременно, на уровне Transformer

HGT — Heterogeneous Graph Transformer

Разные типы создают разные attention head.

Обозначения



$G = (V, E, \phi, \psi)$, где

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

$\phi: V \rightarrow A$ — функция, возвращающая тип узла

$\psi: E \rightarrow R$ — функция, возвращающая тип ребра

Здесь:

- $A = \{a_1, a_2, \dots, a_{|A|}\}$ — множество типов узлов,
- $R = \{r_1, r_2, \dots, r_{|R|}\}$ — множество типов рёбер.

Граф называется гетерогенным, если:

$$|A| > 1 \quad \text{или} \quad |R| > 1.$$

Network Schema

Понятие схемы - ключевое в работах по HIN

(Sun et al., 2013).

Схема гетерогенного графа — это граф второго уровня:

$$S = (A, R)$$

где:

- вершины схемы = типы узлов,
- рёбра схемы = типы отношений.

Пример схемы для рекомендательной системы:

User - view → Item - belongs → Category

Схема задаёт *семантику* связей:

- User взаимодействует с Item,
- Item принадлежит Category — но не наоборот,
- Category не взаимодействует напрямую с User

Метапути (Meta-paths)

Метапуть - это последовательность типов рёбер, связывающая типы узлов:

$$P: A_1 \rightarrow^{R_1} A_2 \rightarrow^{R_2} \dots \rightarrow^{R_k} A_{k+1}$$

Он определяет семантический путь в графе.

Пример из citation graph:

$$Author \rightarrow^{writes} Paper \rightarrow^{written_by} Author$$

Интерпретация метапути: «Два автора похожи, если они пишут статьи совместно»

Другой пример:

$$Paper \rightarrow^{published_in} Venue \rightarrow^{publishes} Paper$$

Интерпретация метапути: «Два статьи похожи, если они публикуются на одном venue»

Метаграфы (Meta-graphs)

Обобщение метапутей.

Метаграф — это структурный шаблон из нескольких метапутей между двумя типами узлов.

Пример:

Author —writes→ Paper —cites→ Paper —written_by→ Author

Этот метаграф описывает:

"Два автора похожи, если один цитирует статьи другого".

Метаграфы дают больше структурной выразительности, чем метапути, но стоят дороже вычислительно.

Семантические пространства

Гетерогенный граф содержит узлы разных типов, чьи признаки:

- имеют разную размерность,
- имеют разную природу (текст, ID, категориальные данные),
- живут в разных пространствах.

Формально:

Признаковое пространство может зависеть от типа узла:

$$h_v^{(0)} \in \mathbb{R}^{d_{\phi(v)}}$$

Где:

- $h_v^{(0)}$ - это начальный признак (feature vector) узла v
- Слой (0) - это просто исходные признаки, до любого GNN-преобразования
- $\phi(v)$ - функция, которая возвращает тип узла
- $d_{\phi(v)}$ - размерность признаков для типа $\phi(v)$

Семантические пространства

Проблема

Message passing в GNN требует, чтобы все соседние узлы преобразовывались совместимо.

Решение: приведение к единому пространству

Обычно используется преобразование:

$$x_v = W_{\phi(v)} h_v^{(0)}, \quad x_v \in \mathbb{R}^d$$

где:

- $h_v^{(0)}$ - это начальный признак (feature vector) узла v
- $\phi(v)$ - функция, которая возвращает тип узла
- $W_{\phi(v)}$ — матрица, зависящая от типа узла,
- все узлы приводятся в единое пространство размерности d

Relation types

Отношения (relation types) как параметрическая сущность

В гетерогенном графе каждое ребро имеет тип.

Это означает:

message passing должен быть разным для разных $r \in R$

Простейшее решение (R-GCN):

$$m_{j \rightarrow i}^{(r)} = W_r h_j$$

Более сложное - relation-specific attention (HGT):

$$\alpha_{i,j}^{(r)} = \frac{(W \tau_i^Q h_i)^\top (W \tau_j^K h_j + W_r^R)}{\sqrt{d}}$$

Таким образом, тип ребра задаёт **отдельный канал взаимодействия**.

Единое пространство и relation-level message passing

После приведения типов узлов к единому пространству, формула message passing принимает вид:

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in R} \sum_{j \in N_r(i)} f_r^{(l)}(h_j^{(l)}, h_i^{(l)}) \right)$$

Суммирование по всем типам рёбер

Это embedding узла i на слое l .

Суммирование по всем соседям узла i только для данного типа r

где

σ - нелинейная функция

f_r - relation-specific message function - оператор, который определяет,

как сосед j влияет на узел i через ребро типа r

Полная модель гетерогенного GNN

Стандартная гетерогенная GNN состоит из 3 слоёв:

1) Type-specific transformation

$$x_v = W_{\phi(v)} h_v^{(0)}$$

2) Relation-specific message passing

$$m_i = \sigma \left(\sum_{r \in R} \sum_{j \in N_r(i)} M_r(x_j, x_i) \right)$$

где M_r - оператор для типа ребра r

3) Semantic aggregation (optional)

(если используются метапути, метаграфы или attention)

$$h_i = \sum_P \beta_P \cdot h_i^P$$

*формула из HAN (Heterogeneous Attention Network)

R-GCN (Relational Graph Convolutional Network)

GCN:

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in N_r(i)} \frac{1}{c_{ij}} W h_j^{(l)} \right)$$

R-GCN:

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in R} \sum_{j \in N_r(i)} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

отдельная матрица весов
для всех соседей,
пришедших по relation r.

Self-loop
трансформация

R-GCN (Relational Graph Convolutional Network)

В R-GCN на каждом relation r есть своя матрица:

$$W_r \in \mathbb{R}^{d \times d}$$

Если:

- $|R|$ — число типов рёбер (в Knowledge Graph оно часто 100–1000)
- d — размер embedding (обычно 100–500)

то модель должна обучать:

$$|R| \cdot d^2$$

Проблема R-GCN: слишком много параметров

Если $|R| = 200$, embedding размер $d = 200$: $200 \times 200^2 = 8\,000\,000$ параметров

Это слишком много.

Чтобы решить проблему, R-GCN вводит **weight decomposition** — разложение больших матриц в компактные формы

R-GCN (Relational Graph Convolutional Network)

Basis decomposition

Вместо того чтобы обучать отдельную матрицу W_r для каждого типа отношения.

Авторы предполагают, что **все relation-матрицы лежат в одном линейном подпространстве**, и потому их можно представить как комбинацию нескольких “базисных” матриц.

$$\underbrace{W_r}_{d \times d} = \sum_{b=1}^B \underbrace{a_{r,b}}_{\text{скаляр}} \cdot \underbrace{V_b}_{d \times d}$$

Где:

V_b - basis matrices. Всего их B штук, $B \ll |R|$

Это общие для всех отношений матрицы.

$a_{r,b}$ - коэффициенты для relation r (обучаются отдельно для каждого r)

Это скаляры, показывающие, насколько relation r “опирается” на basis b .

R-GCN (Relational Graph Convolutional Network)

Число параметров при Basis

Было:

$$|R| \cdot d^2$$

Стало:

$$B \cdot d^2 + |R| \cdot B$$

Если $B = 10$, а $|R|=200$ и $d=200$:

Basis:

$$10 \cdot 200^2 + 200 \cdot 10 = 400\,000 + 2\,000 = 402\,000$$

Оригинальные W_r :

$$200 \times 200^2 = 8\,000\,000$$

20-кратное уменьшение параметров

R-GCN (Relational Graph Convolutional Network)

Block-Diagonal Decomposition (альтернативный подход)

Второй способ уменьшить размерность — не комбинировать всю матрицу, а **разрезать её на блоки**, и обучать только маленькие блоки Q_r

$$W_r = \begin{bmatrix} Q_{r,1} & 0 & \dots & 0 \\ 0 & Q_{r,2} & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & Q_{r,B} \end{bmatrix}$$

Где:

- каждый $Q_{r,b} \in \mathbb{R}^{(d/B) \times (d/B)}$
- полная матрица собирается из блоков

Обычная матрица имеет:

$$d^2$$

Block-diagonal матрица:

$$B \cdot \left(\frac{d}{B}\right)^2 = \frac{d^2}{B}$$

То есть параметров стало меньше в

B раз.

R-GCN для node classification

1. Собираем информацию от соседей с учётом типа связи

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in R} \sum_{j \in N_r(i)} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

То есть каждый сосед j передаёт узлу i информацию, но через матрицу, соответствующую типу связи r .

2. Применяем разложения (Basis или Block)

Если $|R|$ велико, мы заменяем W_r на Basis decomposition или Block-diagonal decomposition

3. Прогоняем несколько слоёв R-GCN

Обычно:

- 2–3 слоя (как GCN),
- Dropout,
- Нормализация.

R-GCN для node classification

4. Финальный слой — классификатор

После L слоёв у каждого узла есть embedding h_i^L .

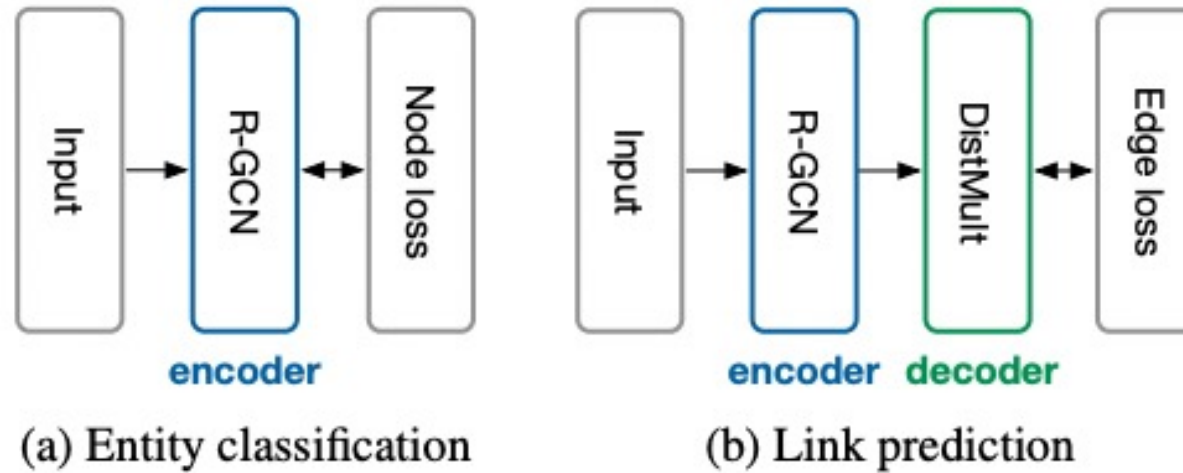
Мы подаём его в softmax:

$$p(y_i | h_i^L) = \text{softmax}(W_c h_i^{(L)})$$

Где:

- W_c — матрица классификатора,
- y_i — метка класса.

R-GCN для link prediction



Источник: <https://arxiv.org/pdf/1703.06103>

HAN (Heterogeneous Attention Network)

Основная идея HAN

HAN (Heterogeneous Attention Network) — это архитектура, которая решает две ключевые задачи:

1. Учитывать сложные семантики (метапути)

Каждый метапуть отражает **отдельный тип смысла** в гетерогенном графе.

2. Находить важные и неважные метапути (semantic attention)

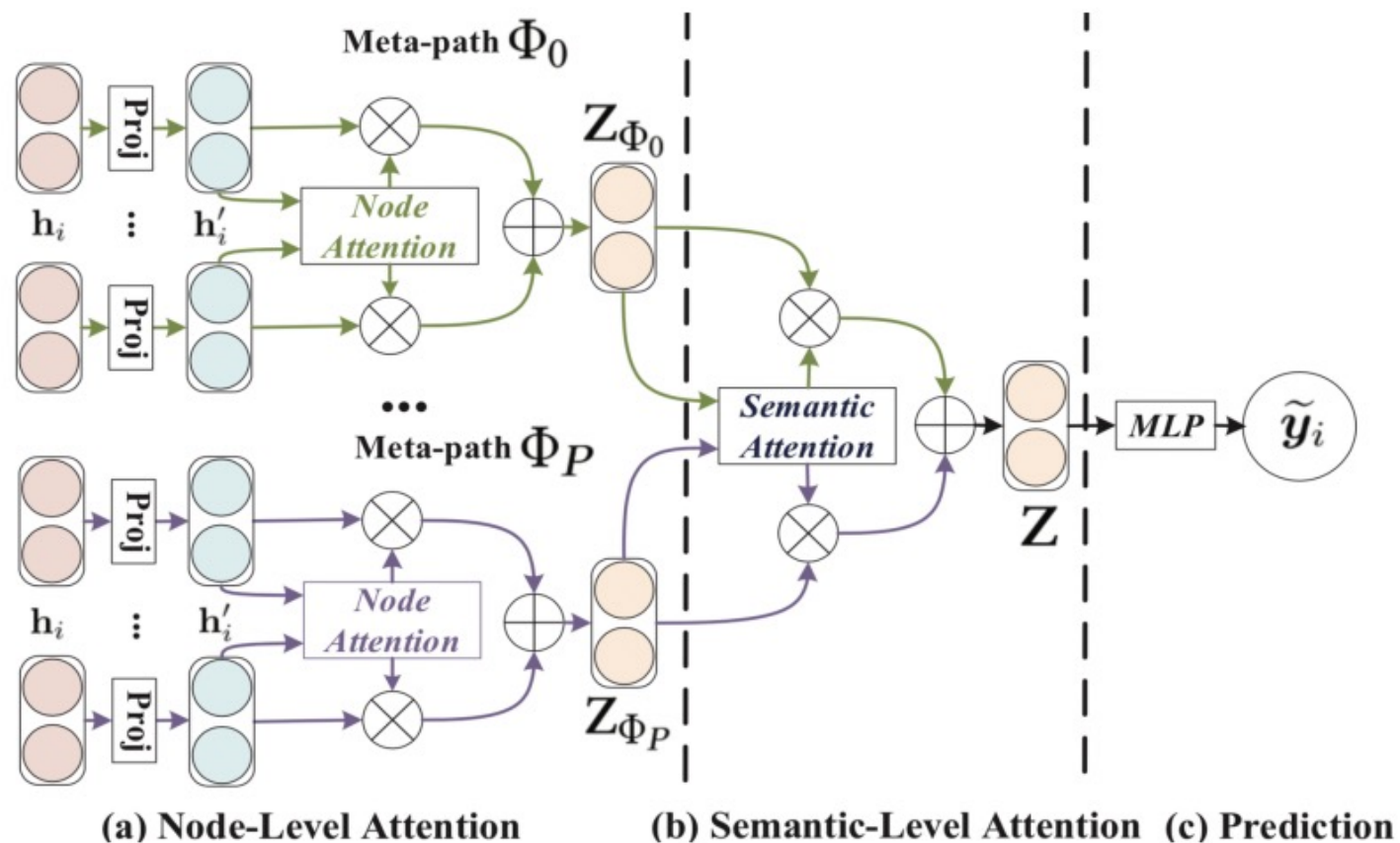
Модель автоматически оценивает:

- какие метапути важны для задачи,
- какие связи сильные,
- какие слабые или шумные.

3. Учитывать внутри-метапутевую структуру (node-level attention)

Attention между узлом и его соседями внутри конкретного метапути.

HAN (Heterogeneous Attention Network)



Источник: <https://arxiv.org/pdf/1903.07293>

HAN (Heterogeneous Attention Network)

Node-level attention

Допустим, у нас есть метапуть P .

Сначала мы строим соседство:

$$N_i^{(P)} = \{j \mid \exists \text{ путь по типу } P\}$$

То есть мы собираем семантических соседей узла i .

Для узла i и его соседа j , относящегося к метапути P :

$$e_{ij}^{(P)} = \text{LeakyReLU}(a^T [Wh_i \parallel Wh_j])$$

Затем attention:

$$\alpha_{ij}^{(P)} = \frac{\exp(e_{ij}^{(P)})}{\sum_{k \in N_i^{(P)}} \exp(e_{ik}^{(P)})}$$

И обновление embedding:

$$h_i^{(P)} = \sigma \left(\sum_{j \in N_i^{(P)}} \alpha_{ij}^{(P)} Wh_j \right)$$

HAN (Heterogeneous Attention Network)

Semantic-level attention

У гетерогенного графа есть **разные типы семантики**, задаваемые метапутями.

Пример (citation network):

- A–P–A : два автора похожи, если пишут вместе
- A–P–V–P–A : два автора похожи, если публикуются на одном venue
- A–I–A : два автора принадлежат одной организации

Каждый метапуть — это **разная семантика**.

«?» - Как объединить несколько семантик в одно итоговое embedding узла?

В HAN появляется **semantic attention**, который автоматически решает:

- какие метапути важнее,
- какие вообще не информативны,
- как их взвесить при формировании финального embedding узла.

HAN (Heterogeneous Attention Network)

Теперь модель должна объединить все эти embedding:

$$h_i^{(APA)}, h_i^{(APVPA)}, h_i^{(AIA)}, \dots = h_i^{(P_1)}, h_i^{(P_2)}, h_i^{(P_3)}, \dots h_i^{(P_M)}$$

где

М — число метапутей.

Нужно объединить их в единое представление узла.

Формула semantic attention:

Сначала считаем важность каждого метапути:

$$w_P = q^T \tanh(W_s h_i^{(P)})$$

Нормируем softmax:

$$\beta_P = \frac{\exp(w_P)}{\sum_{P'} \exp(w_{P'})}$$

И объединяем:

$$h_i = \sum_P \beta_P \cdot h_i^P$$

HAN (Heterogeneous Attention Network)

Итоговая формула:

Мы берём все выбранные
метапути и агрегируем их
вклад

$$h_i = \sum_P \beta_P \cdot h_i^P$$

attention-коэффициент,
показывающий вес
важности метапути

один метапуть

embedding узла i ,
полученный только по
семантике метапути P

Semantic Attention решает: «Какие семантики действительно важны для данной задачи?»

HAN (Heterogeneous Attention Network)

Для каждого метапути P
модель **строит своё собственное embedding узла:**

$$h_i^P$$

Это называется **метапутевой embedding**.

Например:

- $h_i^{(APA)}$ - embedding по пути A–P–A
- $h_i^{(APVPA)}$ - embedding по пути A–P–V–P–A
- $h_i^{(AIA)}$ - embedding по пути A–I–A

Каждый из них получается через attention внутри конкретного метапути.

ИСТОЧНИКИ

1. **“R-GCN — Relational Graph Convolutional Network”** Schlichtkrull et al., *ESWC 2018* - <https://arxiv.org/abs/1703.06103>
2. **“HAN — Heterogeneous Graph Attention Network”** Wang et al., *WWW 2019* - <https://arxiv.org/abs/1903.07293>
3. **“MAGNN — Metapath Aggregated Graph Neural Network”** Fu et al., *CIKM 2020* - <https://arxiv.org/abs/2002.01680>
4. **“GTN — Graph Transformer Networks”** Yun et al., *NeurIPS 2019* - <https://arxiv.org/abs/1911.06455>
5. **“HGT — Heterogeneous Graph Transformer”** Hu et al., *WWW 2020* - <https://arxiv.org/abs/2003.01332>